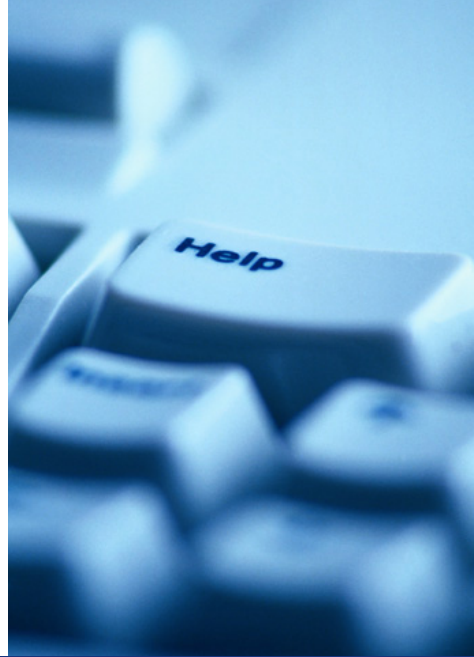




操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



中山大學

SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



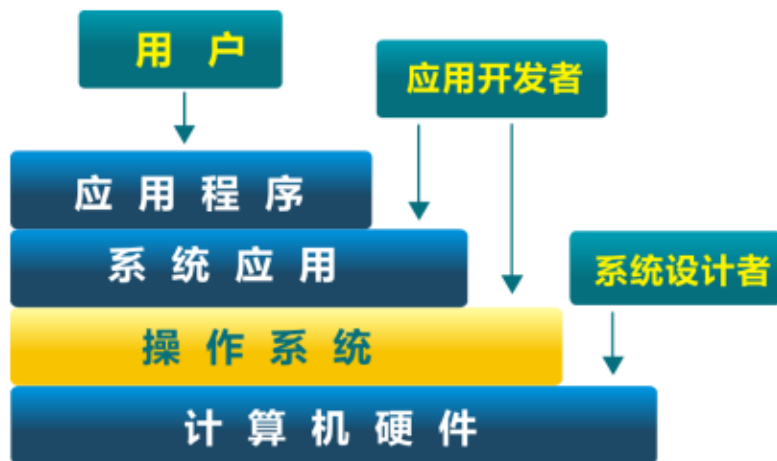
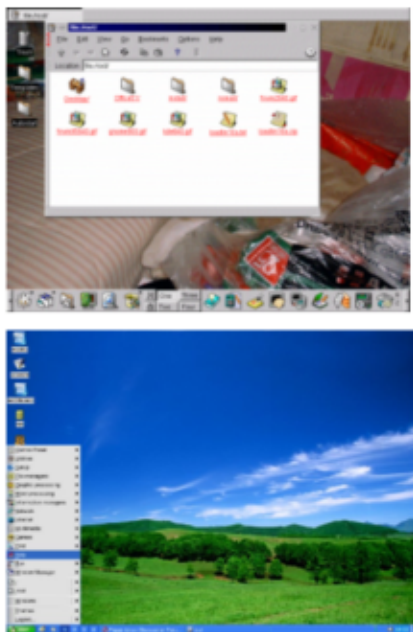
第三讲 — 中断、异常



基本概念和原理

中断 (hardware interrupt)

来自硬件设备 (外设, device) 的处理请求



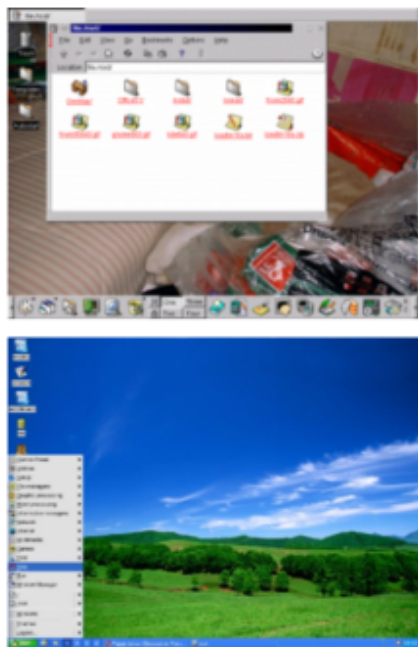
异步：产生原因和当前执行指令无关，如程序被磁盘读打断



基本概念和原理

异常 (exception)

非法指令或者其他原因导致**当前指令执行失败**, (如: 内存出错) 后的处理请求



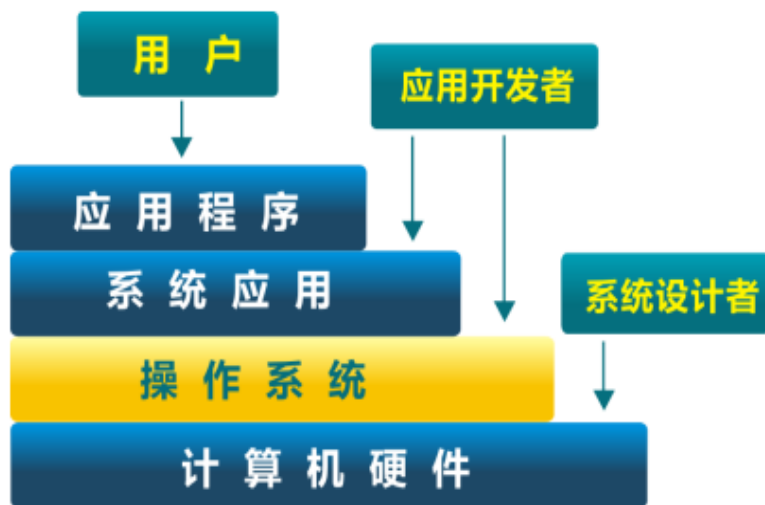
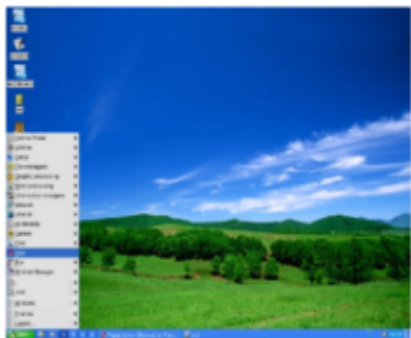
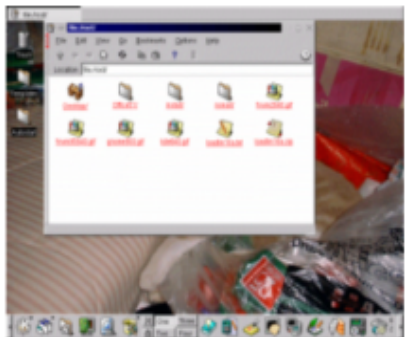
同步：产生和当前执行或试图执行的指令相关



基本概念和原理

系统调用 (system call)

应用程序**主动**向操作系统发出的服务请求



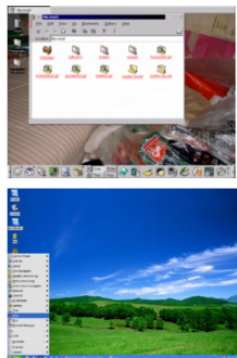


不同体系结构中的中断和异常

通用概念	产生原因	AArch64		x86-64
中断	硬件异步	异常	异步异常 (重置/中断)	中断 (可屏蔽/不可屏蔽)
异常	软件同步		同步异常 (终止/异常指令)	异常 (Fault/Trap/Abort)



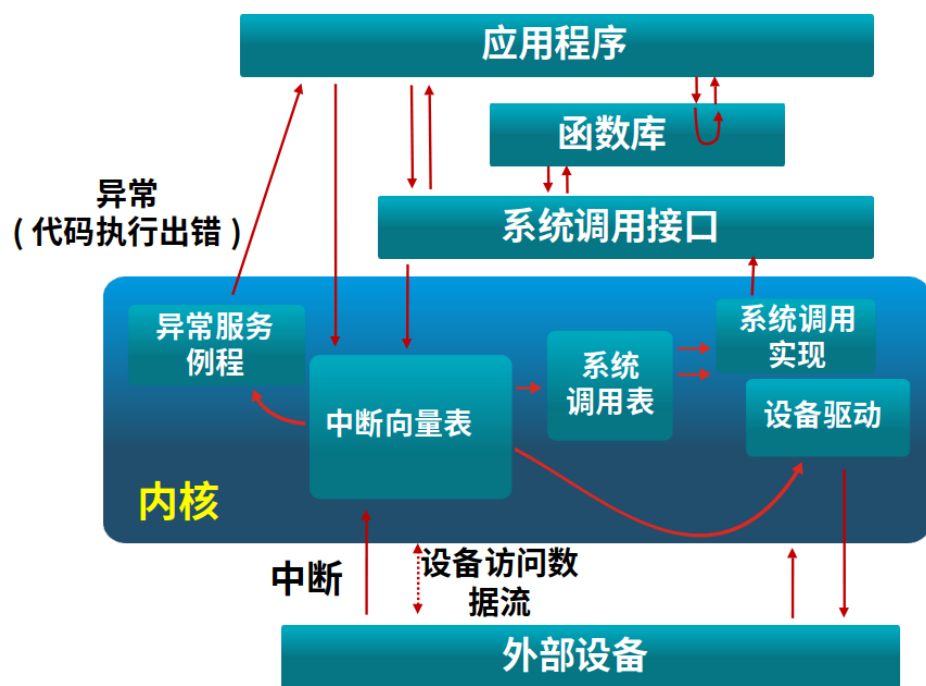
为什么需要



- 为什么需要中断、异常和系统调用
 - OS 内核是被信任的第三方
 - OS 内核可以执行特权指令，管理硬件
 - OS 内核提供了各种 Service
- 中断希望解决的问题
 - 当外设连接计算机时，会出现什么现象？
- 异常希望解决的问题
 - 当应用程序处理意想不到的行为时，会出现什么现象？
- 系统调用希望解决的问题
 - 用户应用程序是如何得到系统服务？



基本概念和原理



● 源头

- 中断：外设
- 异常：应用程序意想不到的行为
- 系统调用：应用程序请求 OS 服务

● 响应方式

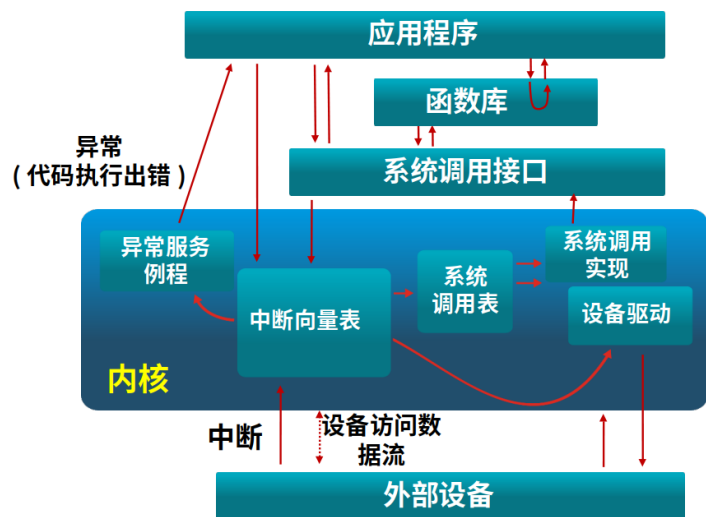
- 中断：异步
- 异常：同步
- 系统调用：异步或同步

● 处理机制

- 中断：持续，对应用程序透明
- 异常：杀死或者重新执行
- 系统调用：等待和持续

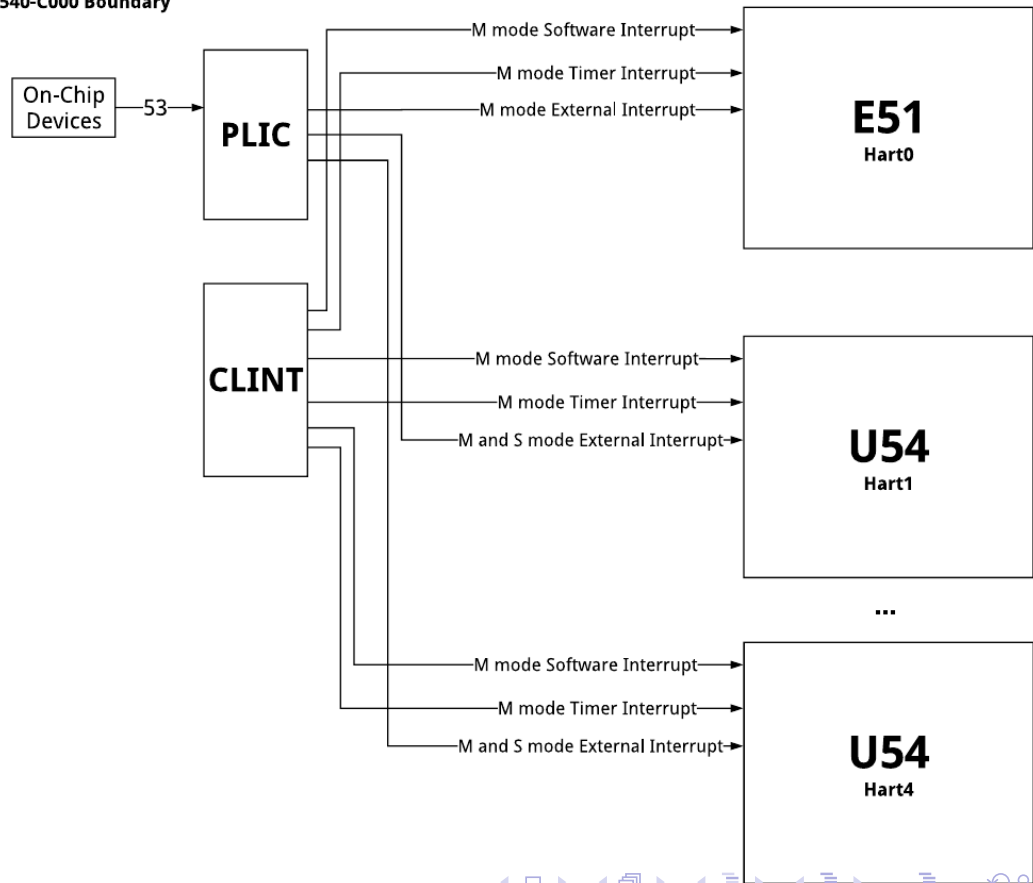


硬件支持



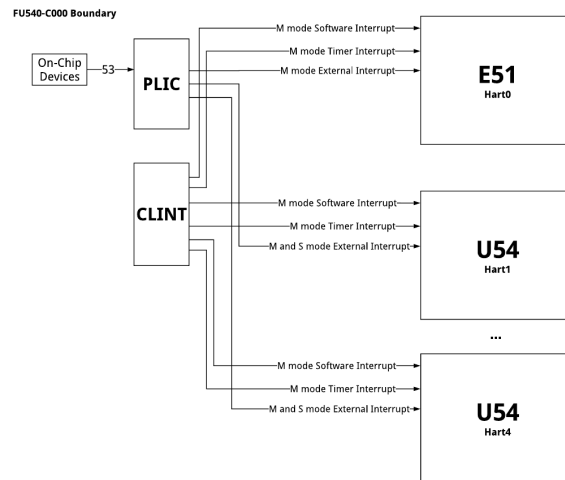
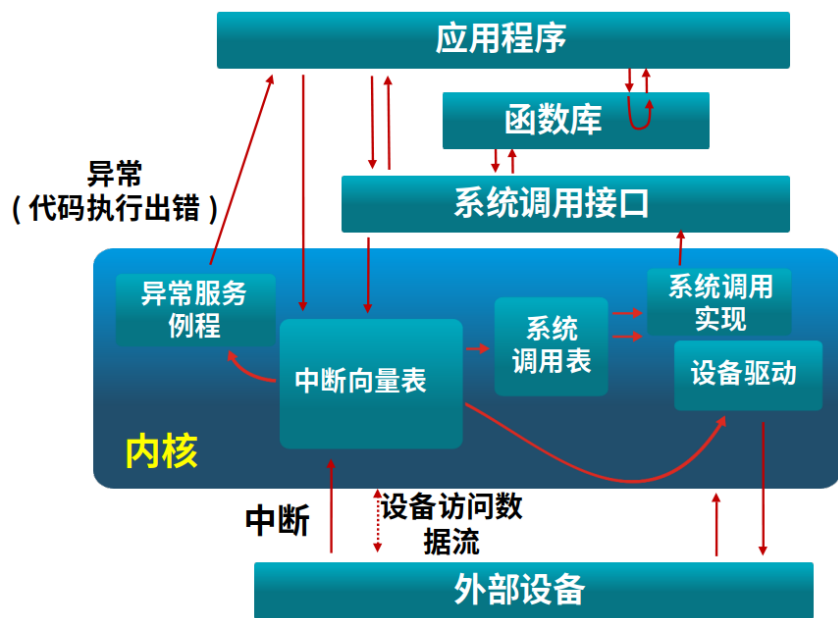
- Core Local Interruptor (CLINT)
- Platform-Level Interrupt Controller (PLIC)

FU540-C000 Boundary





硬件支持



三种标准的中断源:

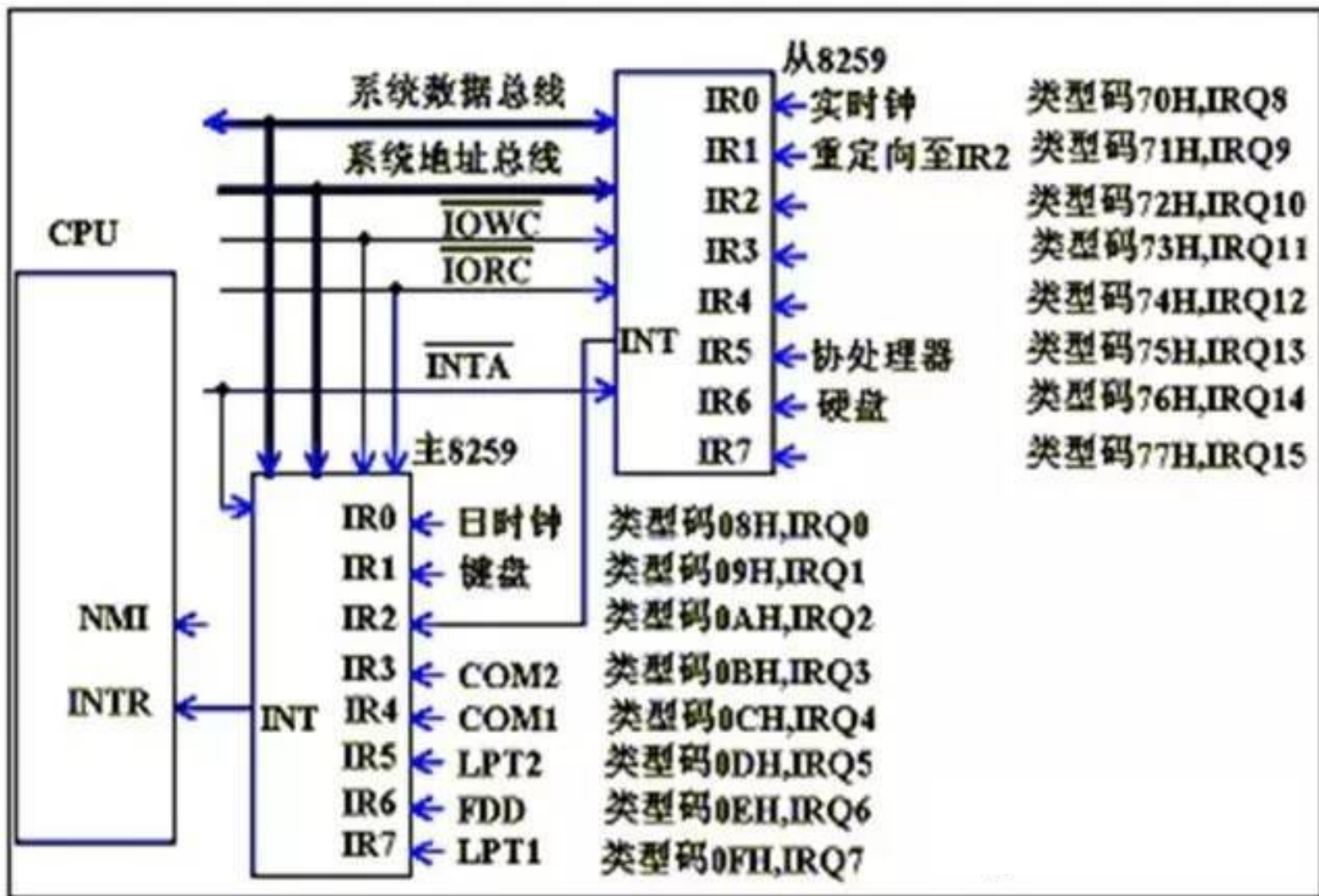
- Core Local Interruptor (CLINT)
- Platform-Level Interrupt Controller (PLIC)

- 软件中断通过向内存映射寄存器中存数来触发, 如 IPI
- 时钟中断, 如 $stimecmp > stime$
- 由平台级中断控制器引发外部中断



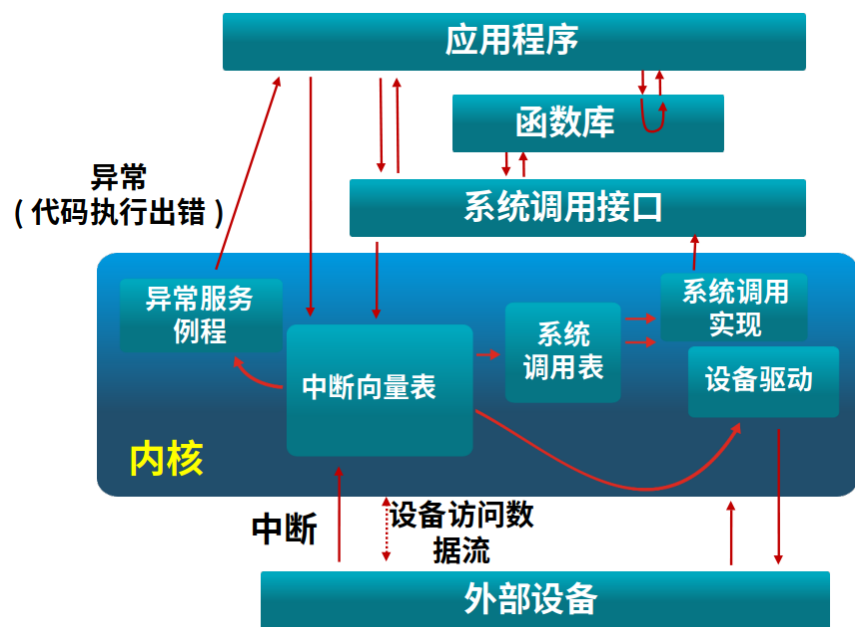
中断处理机制

X86平台 8259中断处理





中断处理机制



- Core Local Interruptor (CLINT)
- Platform-Level Interrupt Controller (PLIC)

建立中断机制

- 建立中断服务例程
- 让 CPU 能响应中断
- 响应并处理中断
- 保存/恢复现场



Linux系统中的中断

❖ 中断 (设备产生、异步)

- 可屏蔽：设备产生的信号，通过中断控制器与处理器相连，可被暂时屏蔽（如，键盘、网络事件）
- 不可屏蔽：一些关键硬件的崩溃（如，内存校验错误）

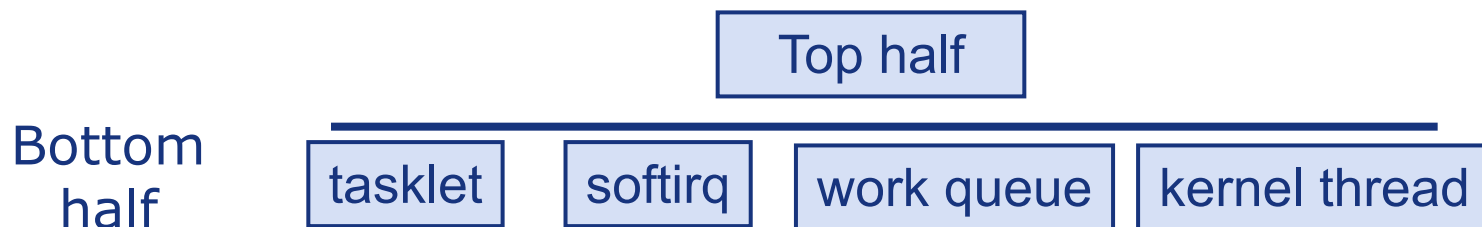
❖ 异常 (软件产生、同步)

- 错误 (Fault)：如缺页异常（可恢复）、段错误（不可恢复）等
- 陷阱 (Trap)：无需恢复，如断点 (int 3)、系统调用 (int 80)
- 中止 (Abort)：严重的错误，不可恢复（机器检查）



Linux系统中的中断

- ❖ 在中断处理中做尽量少的事
- ❖ 推迟非关键行为
- ❖ 结构: Top half & Bottom half
 - Top half : 做最少的工作后返回
 - Bottom half : 推迟处理 (softirq, tasklets, 工作队列, 内核线程)





Top Half: 马上做

❖ 最小的、公共行为

- 保存寄存器、屏蔽其他中断
- 恢复寄存器，返回原来场景

❖ 最重要：调用合适的由硬件驱动提供的中断处理handler

❖ 因为中断被屏蔽，所以不要做太多事情（时间、空间）

❖ 使用将请求放入队列，或者设置标志位将其他处理推迟到 bottom half



Top Half: 找到handler

- ❖ 现代处理器中，多个I/O设备共享一个IRQ和中断向量
- ❖ 多个ISR (interrupt service routines) 可以结合在一个向量上
- ❖ 调用每个设备对应的IRQ的ISR



Bottom Half: 延迟完成

❖ 提供一些推迟完成任务的机制

- softirqs
- tasklets (建立在softirqs之上)
- 工作队列
- 内核线程

❖ 这些工作可以被中断



注意：中断处理没有进程上下文

- ❖ 中断（和异常相比）和具体的某条指令无关
- ❖ 也和中断时正在跑的进程、用户程序无关
- ❖ 中断处理handler不能睡眠！



中断处理中的一些约束条件

- ❖ 不能睡眠
 - 或者调用可能会睡眠的任务
- ❖ 不能调用 `schedule()` 调度
- ❖ 不能释放信号或调用可能睡眠的操作
- ❖ 不能和用户地址空间交换数据



如何安全执行中断

- **Interrupt Vector Table**

- Where the processor looks for a handler
- Limited number of entry points into kernel
- Stored in RAM at a known address

- **Atomic transfer of control**

- Single instruction to change:
 - Program counter
 - Stack pointer
 - Memory protection
 - Kernel/user mode

- **Transparent restartable execution**

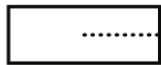
- User program does not know interrupt occurred



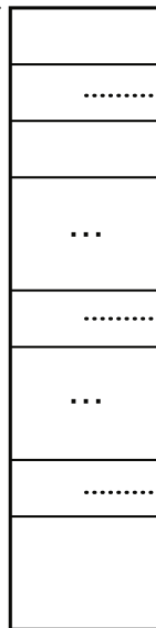
中断向量表

Table set up by OS kernel; pointers to code to run on different events

Processor
Register



Interrupt
Vector



.....→ `handleTimerInterrupt() {`
 `...`
 `}`

.....→ `handleDivideByZero() {`
 `...`
 `}`

.....→ `handleSystemCall() {`
 `...`
 `}`

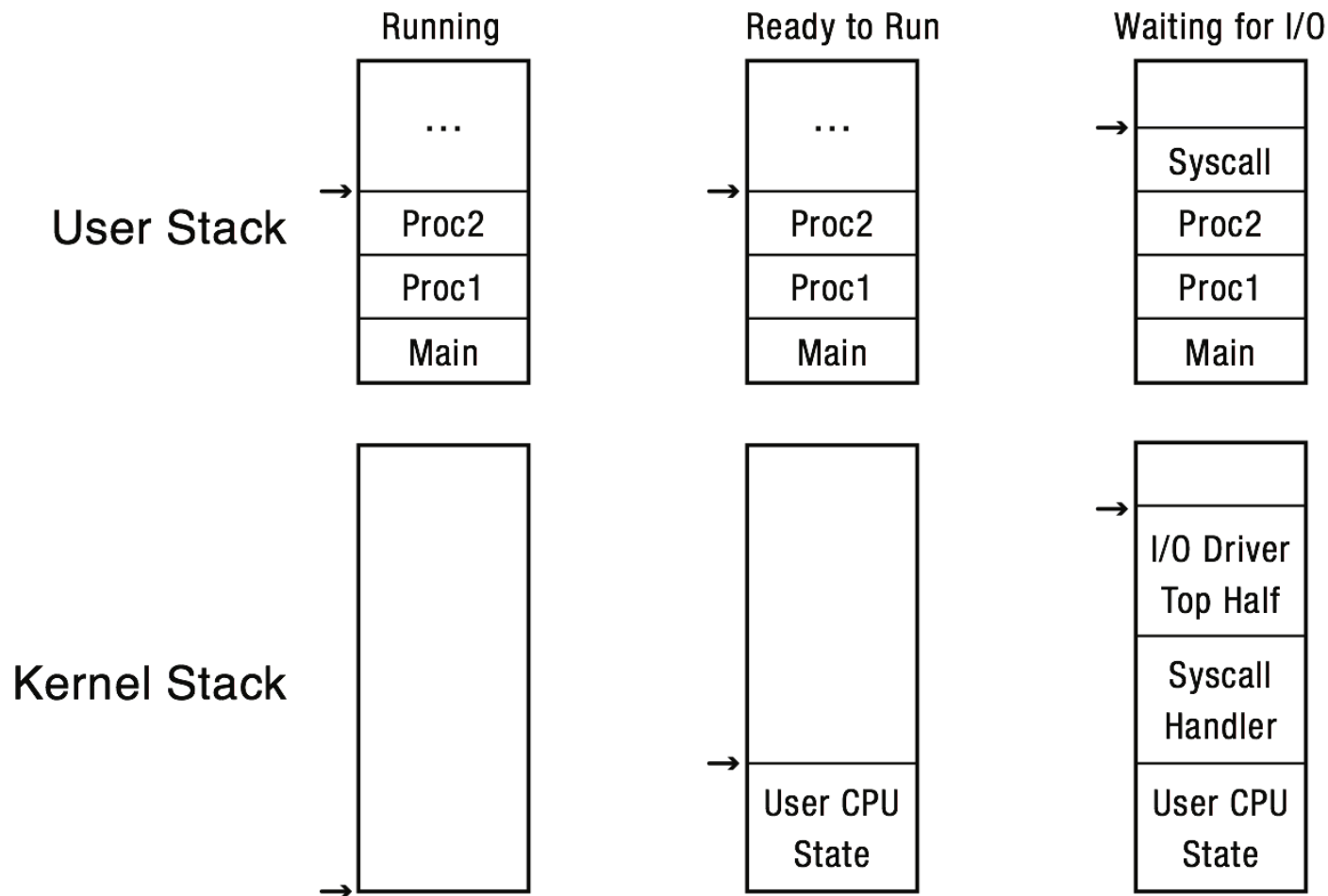


中断向量表

- Per-processor, located in kernel (not user) memory
 - Fun fact! Usually a process/thread has both a kernel and user stack
- **Can the interrupt handler run on the stack of the interrupted user process?**



中断向量表





为什么使用 INT

硬件设备可能需要异步且即时的服务。例如：

- **定时器中断**：定时器和与时间相关的活动需要按照精确的时间间隔随时间推移进行更新。
- **网络中断**：当数据从网络到达时，网卡会向 CPU 发送中断。
- **I/O 设备中断**：当 I/O 设备（如鼠标和键盘）有输入时（例如，一个新字符或鼠标点击），会发出硬件中断。



多核上的中断

多核机器上的中断是如何处理的？

- 在 x86 系统上，每个 CPU 都有自己的本地高级可编程中断控制器（APIC）。它们的连接方式允许将设备中断路由到任何选定的本地 APIC。
- 操作系统可以对 APIC 进行编程，以确定哪些中断被路由到哪些 CPU。
- 默认情况下（除非操作系统另有说明），所有中断都会被路由到处理器 0。



其他类型的中断

- Software Interrupts:
 - Interrupts caused by the execution of a software instruction:
 - `INT <interrupt_number>`
 - Used by the system call `interrupt()`
- Initiated by the running (user level) process
- Cause current processing to be interrupted and transfers control to the corresponding interrupt handler in the kernel



其他类型的中断

- Exceptions
 - Initiated by processor hardware itself
 - Example: divide by zero
- Like a software interrupt, they cause a transfer of control to the kernel to handle the exception



其他类型的中断

- HW -> CPU -> Kernel: Classic HW Interrupt
- User -> Kernel: SW Interrupt
- CPU -> Kernel: Exception
- Interrupt Handlers used in all 3 scenarios



中断屏蔽

- Interrupt handler runs with interrupts off
 - Re-enabled when interrupt completes
- OS kernel can also turn interrupts off
 - Eg., when determining the next process/thread to run



中断处理程序

设计中断处理程序：

- 由于中断处理程序必须尽可能精简，所有与引发中断的事件相关的其他处理工作都必须被延迟执行。
- 示例：
 - 网络中断导致数据包从网卡被复制出来。
 - 对数据包的其他处理工作应延迟到适当的时候再进行。
- 中断处理中被延迟的部分被称为“下半部”（ Bottom Half ）。



Bottom half

延迟部分中断处理的方法

•全局串行化

- 当一个“下半部” (Bottom Half) 正在执行时，任何 CPU 上都不能执行其他“下半部” (即使是不同类型的)。
- 存在明显的性能限制；主要用于提供向后兼容支持。

• **注意**：其他用于延迟工作的机制有时也被称为“下半部”机制。



软中断

- 与“下半部” (Bottom Halves) 类似，这类处理程序必须在编译时在 Linux 内核中被静态定义 / 分配。
-
- 硬件中断处理程序 (在返回之前) 使用 `raise_softirq()` 来标记某个特定的软中断 (`softirq`) 必须执行延迟的工作
- 在稍后的某个时间点，当调度允许时，被标记的软中断处理程序会被执行。
- 当硬件中断完成时
- 当进程发起系统调用时
- 当新进程被调度时

与“下半部”不同，软中断是可重入的，并且可以在多个 CPU 上并发执行。



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢